Tutorial Completo: Microserviços com C# e RabbitMQ

Do Zero ao Deploy — Um Guia Passo a Passo

Sumário

1. Introdução — O que vamos construir?
2. Conceitos Fundamentais
3. Pré-requisitos — O que você precisa instalar
4. Etapa 1 — Criando a Estrutura do Projeto
5. Etapa 2 — OrderService (a API de Pedidos)
6. Etapa 3 — NotificationService (o Worker de Notificações)
7. Etapa 4 — Docker e Docker Compose
8. Etapa 5 — Testes Unitários
9. Etapa 6 — Testes de Integração
10. Etapa 7 — Rodando o Projeto Completo
11. Glossário
12. Diagrama Completo da Arquitetura
13. Dicas e Próximos Passos

1. Introdução — O que vamos construir?

Imagine que você tem uma loja online. Quando um cliente faz um pedido, você precisa:

1. **Registrar o pedido** (anotar que ele existe).
2. **Avisar o cliente** por e-mail, SMS e notificação push no celular.

Agora, imagine que o envio de e-mail demora 3 segundos, o SMS às vezes falha, e a notificação push pode não chegar. Se tudo isso rodar junto no mesmo "clique" do usuário, ele vai ficar esperando uma eternidade — e se o SMS falhar, o pedido inteiro pode dar erro.

A solução? Separar as responsabilidades em **dois programas independentes** (microserviços) que se comunicam por **mensagens**:

```
┌──────────────────┐          ┌────────────────┐          ┌──────────────────────────┐
│  Seu navegador   │──POST──▶│ OrderService │──msg──▶│  NotificationService  │
│  (ou Postman)    │◀──202───│  (API Web)   │        │  (Worker em segundo   │
└──────────────────┘          └────────────────┘          │   plano)              │
                                                          └──────────────────────────┘

                                  ▲                          ▲
                                  │                          │
                                  └──────── RabbitMQ ─────────┘
                                       (o "correio" das
                                        mensagens)
```

OrderService: recebe o pedido do cliente via HTTP, responde "recebido!" instantaneamente, e coloca uma mensagem na fila do RabbitMQ.

NotificationService: fica o tempo todo ouvindo a fila. Quando uma mensagem chega, ele envia e-mail, SMS e push. Se falhar, tenta de novo. Se falhar demais, manda pra uma "fila de mortos" (DLQ) pra alguém resolver depois.

Por que isso é útil?

- **O cliente não espera**: a resposta é instantânea.
- **Se uma notificação falhar, o pedido não falha**: são coisas separadas.
- **Cada serviço pode escalar sozinho**: muitas notificações? Sobe mais instâncias do NotificationService. Muitos pedidos? Escala o OrderService.

2. Conceitos Fundamentais

Antes de colocar a mão no código, vamos entender os conceitos que vamos usar. Pense neles como as peças de um quebra-cabeça — cada uma faz sentido sozinha, mas juntas formam o projeto completo.

2.1 Microserviços

Um **microserviço** é um programa pequeno e independente que faz **uma coisa bem feita**. Em vez de um programa gigante que faz tudo (chamado de "monolito"), você divide em pedaços menores.

Analogia: pense em uma pizzaria. Em vez de uma única pessoa que faz a massa, coloca o recheio, assa e entrega, você tem especialistas: um pizzaiolo, um forno automático e um entregador. Cada um trabalha no seu ritmo.

2.2 RabbitMQ — O Carteiro das Mensagens

O **RabbitMQ** é um programa que funciona como um correio. Seu OrderService escreve uma "carta" (mensagem) e coloca no correio. O NotificationService vai lá e pega a carta quando estiver pronto.

Conceitos do RabbitMQ que vamos usar:

Conceito	O que é	Analogia
Queue (Fila)	Onde as mensagens ficam esperando	A caixa de correio
Exchange	O "roteador" que decide para qual fila a mensagem vai	O carteiro que lê o endereço
Routing Key	O "endereço" na mensagem	O CEP na carta
Binding	A regra que liga uma exchange a uma fila	"Cartas com CEP 01310 vão para a caixa do João"
Producer	Quem envia a mensagem	Quem escreve e envia a carta
Consumer	Quem recebe e processa a mensagem	Quem abre e lê a carta
Ack (Acknowledge)	Confirmação de que a mensagem foi processada	"Li a carta, pode descartar"
DLQ (Dead Letter Queue)	Fila para mensagens que falharam demais	A seção de "cartas perdidas" do correio

2.3 Docker — A Caixa Mágica

O **Docker** empacota seu programa e tudo que ele precisa (sistema operacional, dependências, configurações) em uma "caixa" chamada **container**. Essa caixa roda igual em qualquer computador.

Analogia: é como mandar uma mudança numa caixa fechada. Não importa se a pessoa vai colocar no apartamento ou na casa — dentro da caixa, tudo está organizado do mesmo jeito.

- **Dockerfile**: a receita que ensina como montar a caixa.
- **Docker Compose**: o "maestro" que sobe várias caixas de uma vez (RabbitMQ + OrderService + NotificationService).

2.4 Padrões que vamos usar

Padrão	O que faz	Onde usamos
Strategy	Define uma interface e múltiplas implementações	Handlers de notificação (Email, Push, SMS)
Factory Method assíncrono	Cria objetos com lógica assíncrona no construtor	`RabbitMqPublisher.CreateAsync()`
Retry com Backoff Exponencial	Tenta de novo com esperas cada vez maiores	Filas de retry (5s, 15s, 45s)
Dead Letter Queue	Destino para mensagens que falharam demais	`notifications.dlq`
Idempotência	Garante que processar a mesma mensagem duas vezes não causa problemas	`IdempotencyService`
Injeção de Dependência (DI)	Os objetos recebem suas dependências em vez de criá-las	Todo o projeto usa DI nativo do .NET

2.5 O que é uma API REST?

Uma **API REST** é uma forma de programas se comunicarem pela internet usando HTTP (o mesmo protocolo que seu navegador usa). No nosso caso:

- Seu navegador (ou Postman ou curl) manda uma **requisição HTTP POST** para `http://localhost:5000/orders`
- O OrderService recebe, processa e devolve uma **resposta HTTP 202** ("Aceito, estou cuidando disso")

3. Pré-requisitos — O que você precisa instalar

Antes de começar, instale estas ferramentas. Siga a ordem.

3.1 .NET 8 SDK

O SDK é o kit de desenvolvimento do .NET — ele permite criar, compilar e rodar projetos C#.

1. Acesse: https://dotnet.microsoft.com/download
2. Baixe o **.NET 8 SDK** (não o Runtime — o SDK já inclui o Runtime).
3. Instale normalmente (next, next, finish).
4. Verifique no terminal:

```
dotnet --version
```

Deve mostrar algo como `8.0.xxx`.

3.2 Docker Desktop

O Docker vai rodar o RabbitMQ e os nossos serviços em containers.

1. Acesse: https://www.docker.com/products/docker-desktop
2. Baixe e instale o Docker Desktop.
3. Abra o Docker Desktop e espere ele iniciar (o ícone da baleia na bandeja do sistema deve ficar verde).
4. Verifique no terminal:

```
docker --version
docker compose version
```

3.3 Um editor de código

Recomendo o **Visual Studio Code** ou o **Visual Studio 2022**. Se estiver usando o **Cursor**, melhor ainda.

3.4 Git (opcional, mas recomendado)

Para versionar seu código:

```
git --version
```

Se não tiver, baixe em: https://git-scm.com/downloads

4. Etapa 1 — Criando a Estrutura do Projeto

Vamos criar tudo do zero, comando por comando. Abra seu terminal (PowerShell) e siga.

4.1 Criar a pasta do projeto

```
mkdir microservices-demo
cd microservices-demo
```

4.2 Inicializar o Git

```
git init
```

4.3 Criar o arquivo .gitignore

Crie um arquivo chamado `.gitignore` na raiz do projeto com o seguinte conteúdo:

```
# Build
bin/
obj/
out/

# .NET
*.user
*.suo
.vs/

# Logs
*.log

# Docker
.dockerignore
```

O que é o .gitignore? Ele diz ao Git quais arquivos e pastas ignorar. As pastas `bin/` e `obj/` são geradas automaticamente na compilação — não faz sentido versioná-las.

4.4 Criar a Solution (.sln)

Uma **Solution** é um arquivo que agrupa todos os projetos .NET de uma aplicação. Pense nela como um "guarda-chuva" que contém todos os projetos.

```
dotnet new sln -n microservices-demo
```

Isso cria o arquivo `microservices-demo.sln`.

4.5 Criar o projeto OrderService (API Web)

```
dotnet new webapi -n OrderService -f net8.0 --no-openapi
```

O que esse comando faz?

- `dotnet new webapi` : cria um novo projeto de API Web.
- `-n OrderService` : o nome do projeto.
- `--no-openapi` : não inclui Swagger (para manter simples por enquanto).

4.6 Criar o projeto NotificationService (Worker)

```
dotnet new worker -n NotificationService -f net8.0
```

O que é um Worker? É um tipo de projeto .NET que roda como um serviço em segundo plano. Diferente de uma API (que fica esperando requisições HTTP), o Worker fica rodando continuamente, executando uma tarefa repetitiva — no nosso caso, ouvir mensagens do RabbitMQ.

4.7 Criar o projeto de testes unitários

```
dotnet new xunit -n OrderService.Tests -f net8.0
```

4.8 Criar o projeto de testes de integração

```
dotnet new xunit -n Integration.Tests -f net8.0
```

4.9 Adicionar todos os projetos à Solution

```
dotnet sln add OrderService/OrderService.csproj
dotnet sln add NotificationService/NotificationService.csproj
dotnet sln add OrderService.Tests/OrderService.Tests.csproj
dotnet sln add Integration.Tests/Integration.Tests.csproj
```

4.10 Verificar a estrutura

```
dotnet sln list
```

Deve mostrar:

```
Projeto(s)
----------
OrderService\OrderService.csproj
NotificationService\NotificationService.csproj
OrderService.Tests\OrderService.Tests.csproj
Integration.Tests\Integration.Tests.csproj
```

4.11 Estrutura de pastas até agora

```
microservices-demo/
├── .gitignore
├── microservices-demo.sln
├── OrderService/
│   ├── OrderService.csproj
│   ├── Program.cs
│   └── ... (arquivos gerados pelo template)
├── NotificationService/
│   ├── NotificationService.csproj
│   ├── Program.cs
│   ├── Worker.cs
│   └── ...
├── OrderService.Tests/
│   └── ...
└── Integration.Tests/
    └── ...
```

5. Etapa 2 — OrderService (a API de Pedidos)

O OrderService é o ponto de entrada do nosso sistema. Ele:

1. Recebe um pedido via HTTP POST.
2. Publica uma mensagem no RabbitMQ.
3. Responde "recebido!" ao cliente.

5.1 Instalar o pacote do RabbitMQ

Navegue até a pasta do OrderService e instale o pacote:

```
cd OrderService
dotnet add package RabbitMQ.Client --version 7.2.1
```

O que é o RabbitMQ.Client? É a biblioteca oficial do RabbitMQ para .NET. Ela permite que nosso código C# se conecte ao RabbitMQ e envie/receba mensagens.

Volte para a raiz:

```
cd ..
```

5.2 Verificar o .csproj

Abra o arquivo `OrderService/OrderService.csproj` . Ele deve estar assim:

```xml
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="RabbitMQ.Client" Version="7.2.1" />
  </ItemGroup>

</Project>
```

Entendendo cada linha:

- `Sdk="Microsoft.NET.Sdk.Web"` : indica que é um projeto Web (API).
- `<TargetFramework>net8.0</TargetFramework>` : compila para .NET 8.
- `<Nullable>enable</Nullable>` : ativa avisos de null reference (ajuda a evitar bugs).
- `<ImplicitUsings>enable</ImplicitUsings>` : importa automaticamente os namespaces mais comuns.

5.3 Criar o Model — OrderCreatedMessage

Primeiro, limpe os arquivos gerados pelo template que não vamos usar. Depois, crie a estrutura de pastas:

```
mkdir OrderService/Models
mkdir OrderService/Messaging
mkdir OrderService/Controllers
```

> **Nota:** o template pode já ter criado a pasta `Controllers`. Se sim, o comando vai simplesmente avisar que já existe.

Crie o arquivo `OrderService/Models/OrderCreatedMessage.cs` :

```
namespace OrderService.Models;

public class OrderCreatedMessage
{
    public Guid OrderId { get; set; }
    public string CustomerName { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

O que é esse arquivo?

É o **modelo** da mensagem que vamos publicar no RabbitMQ. Quando alguém faz um pedido, criamos um objeto desse tipo com os dados do pedido e enviamos para a fila.

Explicando cada propriedade:

- `OrderId` : um identificador único (GUID) para o pedido. Exemplo: `a1b2c3d4-e5f6-...`
- `CustomerName` : o nome do cliente.
- `TotalAmount` : o valor total do pedido (usamos `decimal` para dinheiro, nunca `float` ou `double` , porque eles perdem precisão com centavos).
- `CreatedAt` : quando o pedido foi criado.

Por que `= string.Empty` ? Para evitar que a propriedade seja `null` . É uma boa prática em C# moderno — quando o `<Nullable>` está ativado, o compilador avisa se você pode ter um valor nulo.

5.4 Criar a Interface — IRabbitMqPublisher

Crie o arquivo `OrderService/Messaging/IRabbitMqPublisher.cs` :

```
namespace OrderService.Messaging;

public interface IRabbitMqPublisher
{
    Task PublishAsync<T>(T message);
}
```

O que é uma interface?

Uma interface é como um **contrato**. Ela diz: "qualquer classe que me implemente precisa ter o método `PublishAsync` ". Mas ela **não diz como** esse método funciona — isso fica para a classe

concreta.

Por que usar interface?

1. **Testabilidade**: nos testes, podemos criar um "falso" (mock) que finge publicar mensagens sem precisar de RabbitMQ de verdade.
2. **Flexibilidade**: se amanhã quisermos trocar RabbitMQ por Kafka, só precisamos criar uma nova classe que implementa `IRabbitMqPublisher`.

O que é `Task`? `Task` indica que o método é **assíncrono** — ele pode esperar por operações demoradas (como enviar dados pela rede) sem travar o programa.

O que é `<T>`? É um **tipo genérico** — significa que esse método aceita qualquer tipo de objeto. Assim, podemos publicar `OrderCreatedMessage`, `PaymentMessage`, ou qualquer outra classe.

5.5 Criar a Implementação — RabbitMqPublisher

Crie o arquivo `OrderService/Messaging/RabbitMqPublisher.cs`:

```dotnet
using System.Text;
using System.Text.Json;
using RabbitMQ.Client;

namespace OrderService.Messaging;

public class RabbitMqPublisher : IRabbitMqPublisher, IAsyncDisposable
{
    private readonly IConnection _connection;
    private readonly IChannel _channel;
    private const string QueueName = "orders";

    private RabbitMqPublisher(IConnection connection, IChannel channel)
    {
        _connection = connection;
        _channel = channel;
    }

    public static async Task<RabbitMqPublisher> CreateAsync(string host, int port = 5672)
    {
        var factory = new ConnectionFactory { HostName = host, Port = port };
        var connection = await factory.CreateConnectionAsync();
        var channel = await connection.CreateChannelAsync();

        await channel.QueueDeclareAsync(
            queue: QueueName,
            durable: true,
            exclusive: false,
            autoDelete: false
        );

        return new RabbitMqPublisher(connection, channel);
    }

    public async Task PublishAsync<T>(T message)
    {
        var json = JsonSerializer.Serialize(message);
        var body = Encoding.UTF8.GetBytes(json);

        var props = new BasicProperties
        {
            Persistent = true
        };
```

```
        await _channel.BasicPublishAsync(
            exchange: "",
            routingKey: QueueName,
            mandatory: false,
            basicProperties: props,
            body: body
        );
    }

    public async ValueTask DisposeAsync()
    {
        await _channel.DisposeAsync();
        await _connection.DisposeAsync();
    }
}
```

Vamos entender cada parte dessa classe:

O construtor privado

```
private RabbitMqPublisher(IConnection connection, IChannel channel)
```

O construtor é `private` — isso significa que **ninguém de fora pode criar** um `RabbitMqPublisher` usando `new`. Mas por quê?

Porque conectar ao RabbitMQ é uma operação **assíncrona** (usa `await`), e construtores em C# **não podem ser assíncronos**. Então usamos o padrão **Factory Method Assíncrono**.

O Factory Method — CreateAsync

```
public static async Task<RabbitMqPublisher> CreateAsync(string host, int port = 5672)
```

Esse é o método que realmente cria a instância. Ele:

1. Cria uma **ConnectionFactory** apontando para o host do RabbitMQ.
2. Abre uma **conexão** assíncrona (como abrir uma linha telefônica).
3. Cria um **canal** dentro dessa conexão (como uma conversa dentro da linha telefônica — RabbitMQ permite múltiplos canais por conexão).
4. **Declara a fila** `orders` — isso garante que a fila existe. Se já existir, não faz nada (é **idempotente**).

Parâmetros da fila:

- `durable: true` → a fila sobrevive a um restart do RabbitMQ (os dados ficam salvos em disco).
- `exclusive: false` → outros consumidores podem se conectar a essa fila.
- `autoDelete: false` → a fila **não** é apagada quando o último consumidor desconecta.

O PublishAsync

```
public async Task PublishAsync<T>(T message)
```

Esse método:

1. **Serializa** o objeto para JSON (transforma o objeto C# em texto).
2. **Converte** o texto JSON para bytes (porque RabbitMQ trabalha com bytes).
3. Define `Persistent = true` → a mensagem é salva em disco pelo RabbitMQ, sobrevivendo a restarts.
4. **Publica** na fila usando `BasicPublishAsync`:
   - `exchange: ""` → usa a exchange padrão (default exchange), que simplesmente entrega a mensagem na fila cujo nome é igual à routing key.
   - `routingKey: "orders"` → como a exchange é a padrão, a mensagem vai direto para a fila `orders`.

O IAsyncDisposable

```
public async ValueTask DisposeAsync()
```

Implementar `IAsyncDisposable` garante que, quando o serviço parar, a conexão e o canal do RabbitMQ sejam **fechados corretamente**. É como desligar o telefone ao terminar a conversa.

5.6 Criar o Controller — OrdersController

Se houver algum controller de exemplo gerado pelo template, apague-o. Depois, crie o arquivo `OrderService/Controllers/OrdersController.cs`:

```
using Microsoft.AspNetCore.Mvc;
using OrderService.Messaging;
using OrderService.Models;

namespace OrderService.Controllers;

[ApiController]
[Route("[controller]")]
public class OrdersController : ControllerBase
{
    private readonly IRabbitMqPublisher _publisher;
    private readonly ILogger<OrdersController> _logger;

    public OrdersController(IRabbitMqPublisher publisher, ILogger<OrdersController> logger)
    {
        _publisher = publisher;
        _logger = logger;
    }

    [HttpPost]
    public async Task<IActionResult> CreateOrder([FromBody] CreateOrderRequest request)
    {
        var message = new OrderCreatedMessage
        {
            OrderId = Guid.NewGuid(),
            CustomerName = request.CustomerName,
            TotalAmount = request.TotalAmount,
            CreatedAt = DateTime.UtcNow
        };

        await _publisher.PublishAsync(message);

        _logger.LogInformation("Pedido {OrderId} publicado no RabbitMQ", message.OrderId);

        return Accepted(new { message.OrderId, Status = "Pedido recebido e sendo processado" });
    }
}

public record CreateOrderRequest(string CustomerName, decimal TotalAmount);
```

Vamos destrinchar cada parte:

Os atributos do controller

```
[ApiController]
[Route("[controller]")]
```

- `[ApiController]` : ativa validação automática do body da requisição. Se o JSON estiver inválido ou faltando campos, o ASP.NET retorna 400 automaticamente.
- `[Route("[controller]")]` : define que a URL base é o nome do controller sem o sufixo "Controller". Como a classe se chama `OrdersController` , a URL base é `/orders` .

O construtor com Injeção de Dependência

```
public OrdersController(IRabbitMqPublisher publisher, ILogger<OrdersController> logger)
```

O controller **não cria** o publisher nem o logger — ele os **recebe** no construtor. Quem fornece esses objetos é o container de **Injeção de Dependência (DI)** do ASP.NET, que veremos na configuração do `Program.cs` .

Por que isso é bom? Porque o controller não sabe (nem precisa saber) como o publisher funciona internamente. Ele só sabe que existe um método `PublishAsync` . Isso torna o código mais testável e desacoplado.

O método CreateOrder

```
[HttpPost]
public async Task<IActionResult> CreateOrder([FromBody] CreateOrderRequest request)
```

- `[HttpPost]` : esse método responde a requisições HTTP POST.
- `[FromBody]` : o ASP.NET pega o JSON do corpo da requisição e converte para o objeto `CreateOrderRequest` .
- Gera um `Guid` único para o pedido.
- Publica a mensagem no RabbitMQ.
- Retorna HTTP **202 Accepted** (não 200 OK, porque o processamento completo ainda vai acontecer em outro serviço).

O record CreateOrderRequest

```
public record CreateOrderRequest(string CustomerName, decimal TotalAmount);
```

Um `record` é um tipo especial em C# que é **imutável por padrão** e gera automaticamente `Equals`, `GetHashCode` e `ToString`. É perfeito para DTOs (Data Transfer Objects) — objetos que só carregam dados.

5.7 Configurar o Program.cs do OrderService

Substitua todo o conteúdo do arquivo `OrderService/Program.cs` por:

```
using OrderService.Messaging;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

var rabbitHost = builder.Configuration["RabbitMQ:Host"] ?? "localhost";

builder.Services.AddSingleton<IRabbitMqPublisher>(_ =>
    RabbitMqPublisher.CreateAsync(rabbitHost).GetAwaiter().GetResult()
);

var app = builder.Build();
app.MapControllers();
app.Run();
```

Entendendo linha por linha:

1. `WebApplication.CreateBuilder(args)` : cria o builder da aplicação com todas as configurações padrão (logging, configuração, DI container, etc.).
2. `builder.Services.AddControllers()` : registra o sistema de controllers MVC no container de DI. Sem isso, o ASP.NET não saberia que existem controllers.
3. `builder.Configuration["RabbitMQ:Host"] ?? "localhost"` : lê o host do RabbitMQ da configuração. No Docker, a variável de ambiente `RabbitMQ__Host` (com dois underscores) é automaticamente mapeada para `RabbitMQ:Host`. Se não existir, usa `"localhost"` (para rodar localmente sem Docker).
4. `builder.Services.AddSingleton<IRabbitMqPublisher>(...)` : registra o publisher como **Singleton** — uma única instância para toda a vida da aplicação. Isso é eficiente porque manter uma conexão RabbitMQ aberta é mais rápido do que abrir e fechar a cada requisição.

   > **O que é Singleton?** É um padrão de vida (lifetime) onde o container de DI cria **uma única instância** e reutiliza para todas as requisições. Outros padrões são `Scoped` (uma instância por requisição HTTP) e `Transient` (uma nova instância a cada vez que é solicitado).

5. `.GetAwaiter().GetResult()` : como estamos dentro de uma factory síncrona do DI, precisamos "desembrulhar" o `Task` retornado por `CreateAsync` . Em produção, existem formas mais elegantes de fazer isso, mas para nosso demo educacional, funciona perfeitamente.
6. `app.MapControllers()` : mapeia as rotas dos controllers (como `/orders`).
7. `app.Run()` : inicia o servidor e começa a aceitar requisições HTTP.

5.8 Configurar o appsettings.json

O arquivo `OrderService/appsettings.json` fica assim:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Entendendo:

- `LogLevel` : define o nível mínimo de log. `Information` mostra mensagens informativas e acima (Warning, Error). `Microsoft.AspNetCore` é configurado como `Warning` para reduzir o "barulho" de logs internos do ASP.NET.
- `AllowedHosts: "*"` : aceita requisições de qualquer host (para desenvolvimento).

> **Onde está a configuração do RabbitMQ?** Não colocamos no appsettings porque o host vem da variável de ambiente `RabbitMQ__Host` definida no Docker Compose. O fallback para `"localhost"` está no código do `Program.cs` .

6. Etapa 3 — NotificationService (o Worker de Notificações)

O NotificationService é mais complexo que o OrderService. Ele é um **Worker** (serviço em segundo plano) que:

1. Conecta ao RabbitMQ.

2. Ouve a fila `orders`.
3. Para cada pedido, cria 3 notificações (e-mail, push, SMS) e publica na fila `notifications`.
4. Ouve a fila `notifications` e processa cada notificação.
5. Se falhar, faz retry com backoff exponencial (5s, 15s, 45s).
6. Se esgotar os retries, manda para a DLQ (Dead Letter Queue).
7. Garante idempotência (não processa a mesma notificação duas vezes).
8. Coleta métricas a cada 30 segundos.

6.1 Instalar os pacotes

```
cd NotificationService
dotnet add package RabbitMQ.Client --version 7.2.1
dotnet add package Microsoft.Extensions.Hosting --version 8.0.1
cd ..
```

O pacote `Microsoft.Extensions.Hosting` fornece a infraestrutura de `BackgroundService`, logging e DI para workers.

6.2 Criar a estrutura de pastas

```
mkdir NotificationService/Models
mkdir NotificationService/Messaging
mkdir NotificationService/Notifications
mkdir NotificationService/Services
```

6.3 Criar os Models

OrderCreatedMessage (cópia do OrderService)

Crie o arquivo `NotificationService/Models/OrderCreatedMessage.cs`:

```
namespace NotificationService.Models;

public class OrderCreatedMessage
{
    public Guid OrderId { get; set; }
    public string CustomerName { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

> **Por que duplicar?** Cada microserviço é **independente** — ele não referencia o código do outro. Em projetos maiores, você poderia criar um projeto compartilhado (`Shared/Contracts`), mas para um demo com dois serviços, duplicar é mais simples e didático.

NotificationMessage

Crie o arquivo `NotificationService/Models/NotificationMessage.cs` :

```
namespace NotificationService.Models;

public record NotificationMessage
{
    public Guid NotificationId { get; init; } = Guid.NewGuid();
    public Guid OrderId { get; init; }
    public string Type { get; init; } = string.Empty;      // "email" | "push" | "sms"
    public string Recipient { get; init; } = string.Empty;
    public string Content { get; init; } = string.Empty;
    public int RetryCount { get; init; } = 0;
}
```

Por que é `record` e não `class` ?

A diferença principal aqui é a palavra-chave `with` . Records permitem criar **cópias imutáveis** com valores alterados. Exemplo:

```
var original = new NotificationMessage { Type = "email", RetryCount = 0 };
var comRetry = original with { RetryCount = 1 };
```

Isso cria um **novo objeto** com `RetryCount = 1` e todos os outros campos iguais ao original. Usamos isso no mecanismo de retry.

O que é `init`? A keyword `init` permite que a propriedade seja definida apenas durante a inicialização do objeto (no construtor ou na criação com `new`). Depois disso, ela se torna somente leitura.

6.4 Criar a Topologia do RabbitMQ — RabbitMqTopology

Essa classe declara toda a "infraestrutura" de filas e exchanges que o NotificationService precisa.

Crie o arquivo `NotificationService/Messaging/RabbitMqTopology.cs`:

```dotnet
using RabbitMQ.Client;

namespace NotificationService.Messaging;

public static class RabbitMqTopology
{
    public const string NotificationsExchange = "notifications.exchange";
    public const string RetryExchange         = "notifications.retry.exchange";

    public const string NotificationsQueue = "notifications";
    public const string DlqQueue           = "notifications.dlq";

    public static readonly int[] RetryDelaysMs = [5_000, 15_000, 45_000];

    public static async Task DeclareAsync(IChannel channel)
    {
        await channel.ExchangeDeclareAsync(
            exchange: NotificationsExchange,
            type: ExchangeType.Direct,
            durable: true
        );

        await channel.ExchangeDeclareAsync(
            exchange: RetryExchange,
            type: ExchangeType.Direct,
            durable: true
        );

        await channel.QueueDeclareAsync(
            queue: NotificationsQueue,
            durable: true,
            exclusive: false,
            autoDelete: false
        );
        await channel.QueueBindAsync(
            queue: NotificationsQueue,
            exchange: NotificationsExchange,
            routingKey: "notification"
        );

        for (int i = 0; i < RetryDelaysMs.Length; i++)
        {
            var retryQueue = $"notifications.retry.{i + 1}";
```

```
            await channel.QueueDeclareAsync(
                queue: retryQueue,
                durable: true,
                exclusive: false,
                autoDelete: false,
                arguments: new Dictionary<string, object?>
                {
                    ["x-message-ttl"]             = RetryDelaysMs[i],
                    ["x-dead-letter-exchange"]    = NotificationsExchange,
                    ["x-dead-letter-routing-key"] = "notification"
                }
            );
            await channel.QueueBindAsync(
                queue: retryQueue,
                exchange: RetryExchange,
                routingKey: $"retry.{i + 1}"
            );
        }

        await channel.QueueDeclareAsync(
            queue: DlqQueue,
            durable: true,
            exclusive: false,
            autoDelete: false
        );

        await channel.QueueDeclareAsync(
            queue: "orders",
            durable: true,
            exclusive: false,
            autoDelete: false
        );
    }
}
```

Essa é a parte mais complexa do RabbitMQ. Vamos entender o fluxo completo:

Diagrama da topologia

```
                ┌──────────────────────────────┐
                │     notifications.exchange   │
                │         (DIRECT)             │
                └──────────────┬───────────────┘
                               │ routing key: "notification"
                               ▼
                ┌──────────────────────────────┐
                │     notifications (fila)     │◄──── mensagens novas
                │                              │◄──── mensagens que voltaram do retry
                └──────────────┬───────────────┘
                               │ consumer lê
                               ▼
                     ┌──────────────┐
                     │   Sucesso?   │
                     └──────┬───────┘
                       Sim  │   Não
                       │    │
                       ▼    ▼
                    [OK] ┌────────────────┐
                         │ RetryCount < 3? │
                         └────────┬───────┘
                           Sim    │   Não
                           │      │
                           ▼      ▼
   ┌────────────────────────┐  ┌────────────────────┐
   │ notifications.retry.   │  │  notifications.dlq  │
   │ exchange (DIRECT)      │  │  (fila final)       │
   └────────────┬───────────┘  └────────────────────┘
                │ routing key:
                │ "retry.1" / "retry.2" / "retry.3"
                ▼
   ┌──────────────────────────┐
   │ notifications.retry.1    │ ← TTL 5s (espera 5 segundos)
   │ notifications.retry.2    │ ← TTL 15s (espera 15 segundos)
   │ notifications.retry.3    │ ← TTL 45s (espera 45 segundos)
   └──────────────────────────┘
              │ quando o TTL expira, o RabbitMQ
              │ automaticamente move para:
              │ x-dead-letter-exchange = notifications.exchange
              │ x-dead-letter-routing-key = "notification"
              ▼
```

```
┌──────────────────────────┐
│  notifications (fila)    │  ← volta para a fila principal!
└──────────────────────────┘
```

O que é Backoff Exponencial?

Quando uma notificação falha, não tentamos de novo imediatamente. Esperamos:

- **1ª falha**: 5 segundos
- **2ª falha**: 15 segundos (3x mais)
- **3ª falha**: 45 segundos (3x mais)

Se ainda falhar após 3 retries, vai para a DLQ. Isso é chamado de **backoff exponencial** — a espera cresce exponencialmente, dando tempo para o problema (servidor de e-mail fora do ar, por exemplo) se resolver.

Como o TTL funciona?

As filas de retry têm `x-message-ttl` — um tempo máximo que a mensagem pode ficar na fila. Quando esse tempo expira, o RabbitMQ automaticamente move a mensagem para o `x-dead-letter-exchange` com o `x-dead-letter-routing-key` configurado. É assim que a mensagem "volta" para a fila principal após esperar.

6.5 Criar o OrderConsumer

Esse consumer lê pedidos da fila `orders` e gera 3 notificações para cada pedido.

Crie o arquivo `NotificationService/Messaging/OrderConsumer.cs` :

```dotnet
using System.Text;
using System.Text.Json;
using NotificationService.Models;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;

namespace NotificationService.Messaging;

public class OrderConsumer
{
    private readonly IChannel _consumeChannel;
    private readonly IChannel _publishChannel;
    private readonly ILogger _logger;

    public OrderConsumer(IChannel consumeChannel, IChannel publishChannel, ILogger logger)
    {
        _consumeChannel = consumeChannel;
        _publishChannel = publishChannel;
        _logger = logger;
    }

    public async Task StartAsync(CancellationToken ct)
    {
        await _consumeChannel.BasicQosAsync(
            prefetchSize: 0, prefetchCount: 1, global: false, cancellationToken: ct
        );

        var consumer = new AsyncEventingBasicConsumer(_consumeChannel);
        consumer.ReceivedAsync += HandleOrderAsync;

        await _consumeChannel.BasicConsumeAsync(
            queue: "orders",
            autoAck: false,
            consumer: consumer,
            cancellationToken: ct
        );

        _logger.LogInformation("[OrderConsumer] Aguardando pedidos...");
    }

    private async Task HandleOrderAsync(object _, BasicDeliverEventArgs args)
    {
        var json = Encoding.UTF8.GetString(args.Body.ToArray());
```

```
var order = JsonSerializer.Deserialize<OrderCreatedMessage>(json)!;

var notifications = new[]
{
    new NotificationMessage
    {
        OrderId   = order.OrderId,
        Type      = "email",
        Recipient = $"{order.CustomerName.ToLower().Replace(" ", ".")}@example.com",
        Content   = $"Pedido #{order.OrderId} confirmado. Total: {order.TotalAmount:C}"
    },
    new NotificationMessage
    {
        OrderId   = order.OrderId,
        Type      = "push",
        Recipient = $"device-{order.OrderId:N}"[..16],
        Content   = $"Seu pedido de {order.TotalAmount:C} foi recebido!"
    },
    new NotificationMessage
    {
        OrderId   = order.OrderId,
        Type      = "sms",
        Recipient = "+55 11 9 0000-0000",
        Content   = $"Pedido recebido. Total: R$ {order.TotalAmount:F2}"
    }
};

foreach (var notification in notifications)
{
    var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(notification));
    var props = new BasicProperties { Persistent = true };

    await _publishChannel.BasicPublishAsync(
        exchange: RabbitMqTopology.NotificationsExchange,
        routingKey: "notification",
        mandatory: false,
        basicProperties: props,
        body: body
    );
}

_logger.LogInformation(
    "[OrderConsumer] Pedido {OrderId} → {Count} notificações despachadas",
```

```
            order.OrderId,
            notifications.Length
        );

        await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
    }
}
```

Pontos importantes:

- `BasicQosAsync(prefetchCount: 1)` : diz ao RabbitMQ para enviar **uma mensagem por vez** para este consumer. Isso garante que processamos um pedido de cada vez, o que é importante porque cada pedido gera 3 notificações.
- `autoAck: false` : não confirmamos automaticamente o recebimento. A confirmação (`BasicAckAsync`) é feita **manualmente** após processar com sucesso. Se o serviço cair antes do ack, o RabbitMQ reenvia a mensagem para outro consumer (ou para o mesmo quando ele voltar).
- **Canais separados** (`_consumeChannel` e `_publishChannel`): RabbitMQ recomenda usar canais separados para consumir e publicar. Misturar operações no mesmo canal pode causar problemas de performance.
- **Geração do destinatário**: estamos simulando — em produção, os dados viriam de um banco de dados do cliente.

6.6 Criar o NotificationConsumer

Esse é o consumer mais complexo. Ele processa as notificações com retry, DLQ e idempotência.

Crie o arquivo `NotificationService/Messaging/NotificationConsumer.cs` :

```
using System.Diagnostics;
using System.Text;
using System.Text.Json;
using NotificationService.Models;
using NotificationService.Notifications;
using NotificationService.Services;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;

namespace NotificationService.Messaging;

public class NotificationConsumer
{
    private readonly IChannel _consumeChannel;
    private readonly IChannel _publishChannel;
    private readonly SemaphoreSlim _publishLock = new(1, 1);
    private readonly Dictionary<string, INotificationHandler> _handlers;
    private readonly IdempotencyService _idempotency;
    private readonly MetricsService _metrics;
    private readonly ILogger _logger;

    private const int MaxRetries = 3;

    public NotificationConsumer(
        IChannel consumeChannel,
        IChannel publishChannel,
        IEnumerable<INotificationHandler> handlers,
        IdempotencyService idempotency,
        MetricsService metrics,
        ILogger logger)
    {
        _consumeChannel = consumeChannel;
        _publishChannel = publishChannel;
        _handlers       = handlers.ToDictionary(h => h.Type);
        _idempotency    = idempotency;
        _metrics        = metrics;
        _logger         = logger;
    }

    public async Task StartAsync(CancellationToken ct)
    {
        await _consumeChannel.BasicQosAsync(
            prefetchSize: 0, prefetchCount: 5, global: false, cancellationToken: ct
```

```
    );

    var consumer = new AsyncEventingBasicConsumer(_consumeChannel);
    consumer.ReceivedAsync += HandleNotificationAsync;

    await _consumeChannel.BasicConsumeAsync(
        queue: RabbitMqTopology.NotificationsQueue,
        autoAck: false,
        consumer: consumer,
        cancellationToken: ct
    );

    _logger.LogInformation(
        "[NotificationConsumer] Aguardando notificações... (prefetch: 5, max retries: {Max}
        MaxRetries
    );
}

private async Task HandleNotificationAsync(object _, BasicDeliverEventArgs args)
{
    var json    = Encoding.UTF8.GetString(args.Body.ToArray());
    var message = JsonSerializer.Deserialize<NotificationMessage>(json)!;
    var sw      = Stopwatch.StartNew();

    if (_idempotency.AlreadyProcessed(message.NotificationId))
    {
        _logger.LogWarning(
            "[DUPLICATA] Notificação {Id} ({Type}) já processada. Descartando.",
            message.NotificationId, message.Type
        );
        _metrics.RecordDuplicate();
        await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
        return;
    }

    if (!_handlers.TryGetValue(message.Type, out var handler))
    {
        _logger.LogError(
            "[ERRO] Nenhum handler para o tipo '{Type}'. Descartando.", message.Type
        );
        await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
        return;
    }
```

```
    try
    {
        await handler.HandleAsync(message);

        sw.Stop();
        _idempotency.Register(message.NotificationId);
        _metrics.RecordSuccess(message.Type, sw.ElapsedMilliseconds);

        await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
    }
    catch (Exception ex)
    {
        sw.Stop();
        _metrics.RecordFailure(message.Type);

        if (message.RetryCount < MaxRetries)
        {
            var retryNumber = message.RetryCount + 1;
            var delayMs     = RabbitMqTopology.RetryDelaysMs[message.RetryCount];

            _logger.LogWarning(
                "[RETRY {Attempt}/{Max}] {Type} falhou: {Error}. Aguardando {Delay}s.",
                retryNumber, MaxRetries, message.Type, ex.Message, delayMs / 1000
            );

            await PublishToRetryAsync(message with { RetryCount = retryNumber }, retryNumber
        }
        else
        {
            _logger.LogError(
                "[DLQ] {Type} falhou após {Max} tentativas. Notificação {Id} enviada para DL
                message.Type, MaxRetries, message.NotificationId, ex.Message
            );

            _metrics.RecordDlq();
            await PublishToDlqAsync(message, ex.Message);
        }

        await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
    }
}
```

```
    private async Task PublishToRetryAsync(NotificationMessage message, int retryNumber)
    {
        var body  = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(message));
        var props = new BasicProperties { Persistent = true };

        await _publishLock.WaitAsync();
        try
        {
            await _publishChannel.BasicPublishAsync(
                exchange: RabbitMqTopology.RetryExchange,
                routingKey: $"retry.{retryNumber}",
                mandatory: false,
                basicProperties: props,
                body: body
            );
        }
        finally { _publishLock.Release(); }
    }

    private async Task PublishToDlqAsync(NotificationMessage message, string errorReason)
    {
        var envelope = new { message, errorReason, failedAt = DateTime.UtcNow };
        var body     = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(envelope));
        var props    = new BasicProperties { Persistent = true };

        await _publishLock.WaitAsync();
        try
        {
            await _publishChannel.BasicPublishAsync(
                exchange: "",
                routingKey: RabbitMqTopology.DlqQueue,
                mandatory: false,
                basicProperties: props,
                body: body
            );
        }
        finally { _publishLock.Release(); }
    }
}
```

Pontos-chave que merecem atenção:

O SemaphoreSlim

```
private readonly SemaphoreSlim _publishLock = new(1, 1);
```

O `BasicPublishAsync` do RabbitMQ **não é thread-safe** — se dois threads tentarem publicar ao mesmo tempo no mesmo canal, pode dar problema. O `SemaphoreSlim` garante que apenas **uma publicação aconteça por vez** no canal de publicação. Funciona como um cadeado: antes de publicar, "tranca" (`WaitAsync`); depois, "destranca" (`Release`).

O padrão Strategy para handlers

```
_handlers = handlers.ToDictionary(h => h.Type);
```

Recebemos **todos os handlers** como `IEnumerable<INotificationHandler>` (o container de DI injeta os três: email, push, sms) e criamos um dicionário indexado por tipo. Assim, para processar uma notificação do tipo `"email"` , basta fazer:

```
_handlers.TryGetValue(message.Type, out var handler)
```

O fluxo de retry

```
if (message.RetryCount < MaxRetries)
{
    await PublishToRetryAsync(message with { RetryCount = retryNumber }, retryNumber);
}
```

Quando uma notificação falha:

1. Se `RetryCount < 3` , cria uma cópia da mensagem com `RetryCount + 1` e publica na fila de retry correspondente.
2. A fila de retry tem um TTL. Quando o TTL expira, o RabbitMQ automaticamente devolve a mensagem para a fila principal.
3. O consumer processa novamente. Se falhar de novo, incrementa o retry e repete.

Por que Ack mesmo em falha?

```
await _consumeChannel.BasicAckAsync(args.DeliveryTag, multiple: false);
```

Mesmo quando a notificação falha, damos Ack (confirmação). Parece estranho, mas faz sentido: nós mesmos cuidamos do retry publicando uma **nova mensagem** na fila de retry. Se usássemos `Nack` (rejeição), o RabbitMQ recolocaria a mensagem original na fila imediatamente, sem respeitar o backoff.

6.7 Criar a Interface e os Handlers de Notificação

INotificationHandler

Crie o arquivo `NotificationService/Notifications/INotificationHandler.cs`:

```
using NotificationService.Models;

namespace NotificationService.Notifications;

public interface INotificationHandler
{
    string Type { get; }
    Task HandleAsync(NotificationMessage message);
}
```

Cada handler tem um `Type` (ex: `"email"`) que é usado para rotear a mensagem para o handler correto.

EmailNotificationHandler

Crie o arquivo `NotificationService/Notifications/EmailNotificationHandler.cs`:

```
using NotificationService.Models;

namespace NotificationService.Notifications;

public class EmailNotificationHandler : INotificationHandler
{
    private readonly ILogger<EmailNotificationHandler> _logger;

    public string Type => "email";

    public EmailNotificationHandler(ILogger<EmailNotificationHandler> logger) => _logger = logge

    public async Task HandleAsync(NotificationMessage message)
    {
        await Task.Delay(Random.Shared.Next(50, 150));

        if (Random.Shared.NextDouble() < 0.30)
            throw new InvalidOperationException(
                $"Servidor SMTP indisponível para '{message.Recipient}'"
            );

        _logger.LogInformation(
            "[EMAIL] Enviado para {Recipient} | Pedido {OrderId}",
            message.Recipient, message.OrderId
        );
    }
}
```

O que está acontecendo aqui?

Esse handler **simula** o envio de um e-mail:

1. `Task.Delay(Random.Shared.Next(50, 150))` : simula a latência de um servidor SMTP real (entre 50ms e 150ms).
2. `Random.Shared.NextDouble() < 0.30` : **30% de chance de falhar**. Isso é proposital — queremos ver o mecanismo de retry funcionando. Em produção, você usaria uma biblioteca real de e-mail (como SendGrid ou MailKit).

Em produção, esse handler se conectaria a um servidor SMTP ou a uma API como SendGrid e enviaria o e-mail de verdade.

PushNotificationHandler

Crie o arquivo `NotificationService/Notifications/PushNotificationHandler.cs` :

```
using NotificationService.Models;

namespace NotificationService.Notifications;

public class PushNotificationHandler : INotificationHandler
{
    private readonly ILogger<PushNotificationHandler> _logger;

    public string Type => "push";

    public PushNotificationHandler(ILogger<PushNotificationHandler> logger) => _logger = logger;

    public async Task HandleAsync(NotificationMessage message)
    {
        await Task.Delay(Random.Shared.Next(20, 80));

        if (Random.Shared.NextDouble() < 0.20)
            throw new InvalidOperationException(
                $"Token de dispositivo expirado para '{message.Recipient}'"
            );

        _logger.LogInformation(
            "[PUSH] Enviado para {Recipient} | Pedido {OrderId}",
            message.Recipient, message.OrderId
        );
    }
}
```

20% de chance de falhar — simula um token de push notification expirado (acontece quando o usuário desinstala o app).

SmsNotificationHandler

Crie o arquivo `NotificationService/Notifications/SmsNotificationHandler.cs` :

```
using NotificationService.Models;

namespace NotificationService.Notifications;

public class SmsNotificationHandler : INotificationHandler
{
    private readonly ILogger<SmsNotificationHandler> _logger;

    public string Type => "sms";

    public SmsNotificationHandler(ILogger<SmsNotificationHandler> logger) => _logger = logger;

    public async Task HandleAsync(NotificationMessage message)
    {
        await Task.Delay(Random.Shared.Next(100, 300));

        if (Random.Shared.NextDouble() < 0.40)
            throw new InvalidOperationException(
                $"Operadora indisponível para '{message.Recipient}'"
            );

        _logger.LogInformation(
            "[SMS] Enviado para {Recipient} | Pedido {OrderId}",
            message.Recipient, message.OrderId
        );
    }
}
```

40% de chance de falhar — SMS é historicamente o canal menos confiável, por isso tem a maior taxa de falha simulada.

6.8 Criar os Serviços de Suporte

IdempotencyService

Crie o arquivo `NotificationService/Services/IdempotencyService.cs` :

```
using System.Collections.Concurrent;

namespace NotificationService.Services;

public class IdempotencyService
{
    private readonly ConcurrentDictionary<Guid, DateTime> _processed = new();

    public bool AlreadyProcessed(Guid notificationId) =>
        _processed.ContainsKey(notificationId);

    public void Register(Guid notificationId) =>
        _processed.TryAdd(notificationId, DateTime.UtcNow);

    public void Cleanup()
    {
        var cutoff = DateTime.UtcNow.AddHours(-1);
        foreach (var key in _processed.Keys)
            if (_processed.TryGetValue(key, out var processedAt) && processedAt < cutoff)
                _processed.TryRemove(key, out _);
    }
}
```

O que é idempotência?

Uma operação é **idempotente** quando executá-la várias vezes produz o mesmo resultado que executá-la uma vez. Exemplo: enviar o mesmo e-mail duas vezes para o mesmo cliente é ruim. A idempotência evita isso.

Como funciona:

1. Quando processamos uma notificação com sucesso, registramos o `NotificationId` no dicionário.
2. Se a mesma notificação chegar de novo (porque o RabbitMQ reenviou, por exemplo), verificamos se ela já foi processada.
3. Se sim, descartamos sem processar novamente.

`ConcurrentDictionary` : é uma versão do `Dictionary` que é **thread-safe** — múltiplos threads podem ler e escrever ao mesmo tempo sem problemas.

`Cleanup()` : remove entradas com mais de 1 hora para evitar que o dicionário cresça infinitamente. É chamado a cada 30 segundos pelo Worker.

> **Em produção**, você usaria **Redis** em vez de um dicionário em memória, para que a idempotência funcione mesmo com múltiplas instâncias do serviço.

MetricsService

Crie o arquivo `NotificationService/Services/MetricsService.cs` :

```
using System.Collections.Concurrent;

namespace NotificationService.Services;

public class MetricsService
{
    private long _totalProcessed;
    private long _totalFailed;
    private long _totalDuplicates;
    private long _totalSentToDlq;
    private long _totalElapsedMs;
    private long _successCount;

    private readonly ConcurrentDictionary<string, long> _successByType = new();
    private readonly ConcurrentDictionary<string, long> _failureByType = new();

    public void RecordSuccess(string type, long elapsedMs)
    {
        Interlocked.Increment(ref _totalProcessed);
        Interlocked.Add(ref _totalElapsedMs, elapsedMs);
        Interlocked.Increment(ref _successCount);
        _successByType.AddOrUpdate(type, 1, (_, v) => Interlocked.Increment(ref v));
    }

    public void RecordFailure(string type)
    {
        Interlocked.Increment(ref _totalFailed);
        _failureByType.AddOrUpdate(type, 1, (_, v) => Interlocked.Increment(ref v));
    }

    public void RecordDuplicate() => Interlocked.Increment(ref _totalDuplicates);

    public void RecordDlq() => Interlocked.Increment(ref _totalSentToDlq);

    public void LogSummary(ILogger logger)
    {
        var avgMs = _successCount > 0 ? _totalElapsedMs / _successCount : 0;

        var successByType = string.Join(" | ", _successByType
            .Select(kv => $"{kv.Key}: {kv.Value}"));

        var failureByType = string.Join(" | ", _failureByType
            .Select(kv => $"{kv.Key}: {kv.Value}"));
```

```
        logger.LogInformation(
            "=== MÉTRICAS ===" +
            " Processadas: {Processed}" +
            " | Falhas: {Failed}" +
            " | DLQ: {Dlq}" +
            " | Duplicatas: {Dup}" +
            " | Tempo médio: {Avg}ms" +
            " | Sucesso por tipo: [{ByType}]" +
            " | Falha por tipo: [{FailType}]",
            _totalProcessed,
            _totalFailed,
            _totalSentToDlq,
            _totalDuplicates,
            avgMs,
            successByType,
            failureByType
        );
    }
}
```

Por que `Interlocked`?

Múltiplas notificações são processadas em paralelo (lembra do `prefetchCount: 5`?). Se dois threads tentarem incrementar `_totalProcessed` ao mesmo tempo, podemos perder uma contagem. `Interlocked.Increment` garante uma operação **atômica** — ou seja, o incremento é indivisível e thread-safe.

6.9 Criar o Worker

Substitua o conteúdo de `NotificationService/Worker.cs`:

```
using NotificationService.Messaging;
using NotificationService.Notifications;
using NotificationService.Services;
using RabbitMQ.Client;

namespace NotificationService;

public class Worker : BackgroundService
{
    private readonly IEnumerable<INotificationHandler> _handlers;
    private readonly IdempotencyService _idempotency;
    private readonly MetricsService _metrics;
    private readonly ILoggerFactory _loggerFactory;
    private readonly string _rabbitHost;

    public Worker(
        IEnumerable<INotificationHandler> handlers,
        IdempotencyService idempotency,
        MetricsService metrics,
        ILoggerFactory loggerFactory,
        IConfiguration config)
    {
        _handlers      = handlers;
        _idempotency   = idempotency;
        _metrics       = metrics;
        _loggerFactory = loggerFactory;
        _rabbitHost    = config["RabbitMQ:Host"] ?? "localhost";
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        var logger = _loggerFactory.CreateLogger<Worker>();

        await WaitForRabbitMqAsync(logger, stoppingToken);

        var factory = new ConnectionFactory { HostName = _rabbitHost };
        await using var connection = await factory.CreateConnectionAsync(stoppingToken);

        await using var setupChannel        = await connection.CreateChannelAsync(
            cancellationToken: stoppingToken);
        await using var orderConsumeChannel = await connection.CreateChannelAsync(
            cancellationToken: stoppingToken);
        await using var orderPublishChannel = await connection.CreateChannelAsync(
```

```
        cancellationToken: stoppingToken);
    await using var notifConsumeChannel = await connection.CreateChannelAsync(
        cancellationToken: stoppingToken);
    await using var notifPublishChannel = await connection.CreateChannelAsync(
        cancellationToken: stoppingToken);

    await RabbitMqTopology.DeclareAsync(setupChannel);
    logger.LogInformation("Topologia RabbitMQ declarada com sucesso.");

    var orderConsumer = new OrderConsumer(
        orderConsumeChannel,
        orderPublishChannel,
        _loggerFactory.CreateLogger<OrderConsumer>()
    );
    await orderConsumer.StartAsync(stoppingToken);

    var notifConsumer = new NotificationConsumer(
        notifConsumeChannel,
        notifPublishChannel,
        _handlers,
        _idempotency,
        _metrics,
        _loggerFactory.CreateLogger<NotificationConsumer>()
    );
    await notifConsumer.StartAsync(stoppingToken);

    _ = Task.Run(async () =>
    {
        using var timer = new PeriodicTimer(TimeSpan.FromSeconds(30));
        while (await timer.WaitForNextTickAsync(stoppingToken))
        {
            _metrics.LogSummary(logger);
            _idempotency.Cleanup();
        }
    }, stoppingToken);

    logger.LogInformation("NotificationService pronto. Aguardando mensagens...");
    await Task.Delay(Timeout.Infinite, stoppingToken);
}

private async Task WaitForRabbitMqAsync(ILogger logger, CancellationToken ct)
{
    var factory = new ConnectionFactory { HostName = _rabbitHost };
```

```
        while (!ct.IsCancellationRequested)
        {
            try
            {
                await using var connection = await factory.CreateConnectionAsync(ct);
                logger.LogInformation("Conectado ao RabbitMQ!");
                return;
            }
            catch
            {
                logger.LogWarning("RabbitMQ não está pronto. Tentando novamente em 3s...");
                await Task.Delay(3_000, ct);
            }
        }
    }
}
```

O que é `BackgroundService` ?

É uma classe base do .NET para criar serviços que rodam em segundo plano. Você sobrescreve o método `ExecuteAsync` e ele fica rodando enquanto a aplicação estiver viva.

Entendendo o `ExecuteAsync` :

1. **`WaitForRabbitMqAsync`** : espera o RabbitMQ ficar disponível. No Docker, o RabbitMQ pode demorar alguns segundos para iniciar, então o Worker fica tentando conectar a cada 3 segundos.
2. **5 canais separados**: cada operação (setup, consumo de orders, publicação de notificações, consumo de notificações, publicação de retries) usa seu próprio canal para evitar interferência.
3. **`PeriodicTimer`** : a cada 30 segundos, loga um resumo de métricas e limpa entradas antigas do serviço de idempotência.
4. **`Task.Delay(Timeout.Infinite, stoppingToken)`** : mantém o método rodando indefinidamente até receber o sinal de parada.

6.10 Configurar o Program.cs do NotificationService

Substitua o conteúdo de `NotificationService/Program.cs` :

```
using NotificationService;
using NotificationService.Notifications;
using NotificationService.Services;

var builder = Host.CreateApplicationBuilder(args);

builder.Services.AddSingleton<INotificationHandler, EmailNotificationHandler>();
builder.Services.AddSingleton<INotificationHandler, PushNotificationHandler>();
builder.Services.AddSingleton<INotificationHandler, SmsNotificationHandler>();

builder.Services.AddSingleton<IdempotencyService>();
builder.Services.AddSingleton<MetricsService>();

builder.Services.AddHostedService<Worker>();

var host = builder.Build();
host.Run();
```

Registrando múltiplas implementações:

```
builder.Services.AddSingleton<INotificationHandler, EmailNotificationHandler>();
builder.Services.AddSingleton<INotificationHandler, PushNotificationHandler>();
builder.Services.AddSingleton<INotificationHandler, SmsNotificationHandler>();
```

O container de DI permite registrar **múltiplas implementações** para a mesma interface. Quando o Worker pede `IEnumerable<INotificationHandler>`, o DI injeta **todos os três** handlers. Essa é a base do padrão **Strategy**.

Por que tudo é Singleton aqui?

Cada serviço tem uma razão diferente para ser Singleton:

Serviço	Por que Singleton?
`IRabbitMqPublisher`	Conexão TCP com RabbitMQ é cara de criar. Uma por app é o padrão recomendado.
`EmailHandler`, `PushHandler`, `SmsHandler`	São **stateless** — não guardam nenhum estado mutável. Poderiam ser `Transient`, mas Singleton evita re-instanciação desnecessária.
`IdempotencyService`	**Obrigatório.** Guarda um `ConcurrentDictionary` em memória. Se fosse `Transient`, cada instância teria seu próprio dicionário vazio

Serviço	Por que Singleton?
	— a idempotência não funcionaria.
`MetricsService`	**Obrigatório.** Tem contadores com `Interlocked`. Se fosse `Transient`, cada instância zeraria os contadores — as métricas seriam sempre 0.

Regra geral para escolher o lifetime:

- Tem **estado compartilhado** (dicionário, contadores)? → **precisa** ser Singleton
- Tem **recurso caro** (conexão de rede, pool)? → **deve** ser Singleton
- É **stateless** (só processa e retorna)? → pode ser qualquer um; Singleton é uma escolha razoável

> **Cuidado:** nunca injete um serviço `Scoped` dentro de um `Singleton`. O container de DI do .NET lança uma exceção em tempo de execução se isso acontecer. Neste projeto não temos esse problema, mas vale saber.

6.11 Configurar o appsettings.json

O arquivo `NotificationService/appsettings.json`:

```json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

7. Etapa 4 — Docker e Docker Compose

7.1 Dockerfile do OrderService

Crie o arquivo `OrderService/Dockerfile`:

```dockerfile
# Estágio 1: build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app

COPY *.csproj .
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /out

# Estágio 2: runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /out .

ENTRYPOINT ["dotnet", "OrderService.dll"]
```

O que é um Dockerfile multi-stage?

O Dockerfile tem **dois estágios**:

Estágio 1 (build): usa a imagem `sdk:8.0` (~700 MB), que tem todas as ferramentas de compilação. Copia o código, restaura pacotes e compila.

Estágio 2 (runtime): usa a imagem `aspnet:8.0` (~200 MB), que é muito menor e tem apenas o necessário para **executar** o aplicativo. Copia apenas os arquivos compilados do estágio anterior.

Por que dois estágios? A imagem final fica muito menor (não inclui o SDK de compilação), o que significa downloads mais rápidos e containers mais leves.

Entendendo cada instrução:

Instrução	O que faz
`FROM ... AS build`	Define a imagem base e dá um nome ao estágio
`WORKDIR /app`	Define o diretório de trabalho dentro do container
`COPY *.csproj .`	Copia só o .csproj primeiro (para cachear o restore)
`RUN dotnet restore`	Baixa os pacotes NuGet
`COPY . .`	Copia todo o código fonte

Instrução	O que faz
`RUN dotnet publish -c Release -o /out`	Compila em modo Release e coloca o resultado em /out
`COPY --from=build /out .`	Copia os arquivos compilados do estágio "build"
`ENTRYPOINT [...]`	Comando que roda quando o container inicia

Dica de performance: copiamos o `.csproj` e fazemos `restore` ANTES de copiar o resto do código. Assim, o Docker cacheia a camada do restore. Se só o código mudar (sem alterar dependências), o restore não precisa ser refeito.

7.2 Dockerfile do NotificationService

Crie o arquivo `NotificationService/Dockerfile` :

```
# Estágio 1: build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app

COPY *.csproj .
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /out

# Estágio 2: runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /out .

ENTRYPOINT ["dotnet", "NotificationService.dll"]
```

É praticamente idêntico ao do OrderService — a única diferença é o nome da DLL no `ENTRYPOINT` .

7.3 Docker Compose — Orquestrando tudo

Crie o arquivo `docker-compose.yml` na **raiz** do projeto:

```yaml
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - "5672:5672"   # porta AMQP (sua app conecta aqui)
      - "15672:15672" # painel web de admin
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest

  order-service:
    build: ./OrderService
    depends_on:
      - rabbitmq
    ports:
      - "5000:8080"   # acessa a API em http://localhost:5000
    environment:
      - RabbitMQ__Host=rabbitmq

  notification-service:
    build: ./NotificationService
    depends_on:
      - rabbitmq
    environment:
      - RabbitMQ__Host=rabbitmq
```

O que é Docker Compose?

Docker Compose permite definir e rodar **múltiplos containers** com um único comando. Em vez de rodar `docker run` para cada serviço, escrevemos um arquivo YAML que descreve tudo e rodamos `docker compose up`.

Entendendo cada serviço:

rabbitmq

```yaml
rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"
      - "15672:15672"
```

- `image: rabbitmq:3-management` : usa a imagem oficial do RabbitMQ com o painel web de administração.
- `5672:5672` : porta AMQP — é por aqui que os serviços .NET se conectam.
- `15672:15672` : porta do painel web — acesse http://localhost:15672 para ver filas, mensagens, etc.

order-service

```yaml
order-service:
    build: ./OrderService
    depends_on:
      - rabbitmq
    ports:
      - "5000:8080"
    environment:
      - RabbitMQ__Host=rabbitmq
```

- `build: ./OrderService` : constrói a imagem usando o Dockerfile em `./OrderService` .
- `depends_on: rabbitmq` : garante que o RabbitMQ inicia **antes** do OrderService (mas não espera ele ficar "pronto" — por isso temos o `WaitForRabbitMqAsync`).
- `"5000:8080"` : o ASP.NET roda na porta 8080 dentro do container. Mapeamos para 5000 no host. Então você acessa `http://localhost:5000` .
- `RabbitMQ__Host=rabbitmq` : variável de ambiente que diz ao serviço para conectar no host `rabbitmq` (que é o nome do serviço Docker — Docker Compose cria uma rede interna onde os serviços se encontram pelo nome).

> **Detalhe importante:** no Docker Compose, os serviços se comunicam pelo **nome do serviço** (como DNS). Então `rabbitmq` resolve para o IP interno do container do RabbitMQ. Mágico, não?

> **Por que dois underscores (`__`)?** O ASP.NET mapeia variáveis de ambiente com `__` para `:` na configuração. Então `RabbitMQ__Host` vira `RabbitMQ:Host` , que é lido por `builder.Configuration["RabbitMQ:Host"]` .

notification-service

```yaml
notification-service:
    build: ./NotificationService
    depends_on:
      - rabbitmq
    environment:
      - RabbitMQ__Host=rabbitmq
```

Mesma lógica, mas sem `ports` — o NotificationService não expõe HTTP (ele é um Worker, não uma API).

8. Etapa 5 — Testes Unitários

Testes são essenciais. Vamos criar testes para o `OrdersController`.

8.1 Instalar os pacotes

```bash
cd OrderService.Tests
dotnet add package Moq --version 4.20.72
dotnet add reference ../OrderService/OrderService.csproj
cd ..
```

Pacotes:

- `Moq`: biblioteca de mocking — permite criar "falsificações" de interfaces para testes.
- A referência ao `OrderService.csproj` permite acessar as classes do OrderService nos testes.

8.2 Verificar o .csproj dos testes

O arquivo `OrderService.Tests/OrderService.Tests.csproj` deve ficar assim:

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.0" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.8.0" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <PackageReference Include="xunit" Version="2.5.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.5.3" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\OrderService\OrderService.csproj" />
  </ItemGroup>

</Project>
```

8.3 Criar os testes

Crie a pasta e o arquivo `OrderService.Tests/Controllers/OrdersControllerTests.cs` :

```
mkdir OrderService.Tests/Controllers
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging.Abstractions;
using Moq;
using OrderService.Controllers;
using OrderService.Messaging;
using OrderService.Models;

namespace OrderService.Tests.Controllers;

public class OrdersControllerTests
{
    private readonly Mock<IRabbitMqPublisher> _publisherMock = new();
    private readonly OrdersController _controller;

    public OrdersControllerTests()
    {
        _controller = new OrdersController(
            _publisherMock.Object,
            NullLogger<OrdersController>.Instance
        );
    }

    [Fact]
    public async Task CreateOrder_DeveRetornar202_QuandoRequestValido()
    {
        var request = new CreateOrderRequest("João Silva", 150.90m);

        var result = await _controller.CreateOrder(request);

        Assert.IsType<AcceptedResult>(result);
    }

    [Fact]
    public async Task CreateOrder_DevePublicarMensagemNoRabbitMQ()
    {
        var request = new CreateOrderRequest("João Silva", 150.90m);

        await _controller.CreateOrder(request);

        _publisherMock.Verify(
            p => p.PublishAsync(It.IsAny<OrderCreatedMessage>()),
            Times.Once
        );
```

```
    }

    [Fact]
    public async Task CreateOrder_MensagemPublicada_DeveTerDadosCorretos()
    {
        var request = new CreateOrderRequest("Maria Costa", 299.99m);
        OrderCreatedMessage? mensagemCapturada = null;

        _publisherMock
            .Setup(p => p.PublishAsync(It.IsAny<OrderCreatedMessage>()))
            .Callback<OrderCreatedMessage>(msg => mensagemCapturada = msg);

        await _controller.CreateOrder(request);

        Assert.NotNull(mensagemCapturada);
        Assert.Equal("Maria Costa", mensagemCapturada.CustomerName);
        Assert.Equal(299.99m, mensagemCapturada.TotalAmount);
        Assert.NotEqual(Guid.Empty, mensagemCapturada.OrderId);
    }

    [Fact]
    public async Task CreateOrder_ResponseBody_DeveConterOrderId()
    {
        var request = new CreateOrderRequest("Carlos Lima", 50m);

        var result = await _controller.CreateOrder(request) as AcceptedResult;

        var body = result!.Value!;
        var orderId = body.GetType().GetProperty("OrderId")?.GetValue(body);
        var status = body.GetType().GetProperty("Status")?.GetValue(body) as string;

        Assert.NotNull(orderId);
        Assert.Equal("Pedido recebido e sendo processado", status);
    }
}
```

Entendendo os conceitos de teste:

O que é Mock?

Um **Mock** é um objeto "falso" que simula o comportamento de um objeto real. No nosso caso, `Mock<IRabbitMqPublisher>` cria um publisher falso que não conecta no RabbitMQ de verdade. Isso nos permite testar o controller **isoladamente**, sem precisar de infraestrutura externa.

O que é `[Fact]` ?

É um atributo do xUnit que marca um método como um **teste**. Quando rodamos `dotnet test` , o framework encontra todos os métodos marcados com `[Fact]` e os executa.

Padrão AAA (Arrange, Act, Assert)

Cada teste segue o padrão:

1. **Arrange** (Preparar): criar os objetos e condições necessárias.
2. **Act** (Agir): executar a ação que queremos testar.
3. **Assert** (Verificar): checar se o resultado é o esperado.

Explicando cada teste

Teste 1 — Deve retornar 202:
Verifica que o controller retorna HTTP 202 Accepted quando recebe um request válido.

Teste 2 — Deve publicar no RabbitMQ:
Verifica que o método `PublishAsync` foi chamado **exatamente uma vez**. O `Verify` do Moq é como perguntar: "Ei mock, alguém chamou esse método? Quantas vezes?"

Teste 3 — Dados corretos na mensagem:
Usa `Callback` para **capturar** o argumento passado ao `PublishAsync` e verifica que os dados do pedido estão corretos na mensagem publicada.

Teste 4 — Response body com OrderId:
Verifica que o corpo da resposta contém o `OrderId` e o status correto. Usa reflection (`GetProperty`) porque o controller retorna um objeto anônimo.

NullLogger

```
NullLogger<OrdersController>.Instance
```

Um logger que **descarta tudo** — nos testes, não precisamos ver logs.

9. Etapa 6 — Testes de Integração

Os testes de integração verificam que os serviços funcionam juntos com infraestrutura real. Usamos **Testcontainers** para subir um RabbitMQ de verdade em um container Docker durante os testes.

9.1 Instalar os pacotes

```
cd Integration.Tests
dotnet add package RabbitMQ.Client --version 7.2.1
dotnet add package Testcontainers.RabbitMq --version 4.11.0
dotnet add reference ../OrderService/OrderService.csproj
cd ..
```

O que é Testcontainers?

É uma biblioteca que cria e gerencia containers Docker durante os testes. Ela:

1. Sobe um container RabbitMQ antes do teste.
2. Fornece a porta mapeada para conectar.
3. Derruba o container automaticamente após o teste.

É como ter um RabbitMQ temporário e descartável só para testes.

9.2 Verificar o .csproj

O `Integration.Tests/Integration.Tests.csproj`：

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.0" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.8.0" />
    <PackageReference Include="RabbitMQ.Client" Version="7.2.1" />
    <PackageReference Include="Testcontainers.RabbitMq" Version="4.11.0" />
    <PackageReference Include="xunit" Version="2.5.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.5.3" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\OrderService\OrderService.csproj" />
  </ItemGroup>

</Project>
```

9.3 Criar os testes de integração

Crie o arquivo `Integration.Tests/OrderCreatedFlowTests.cs`:

```
using System.Text;
using System.Text.Json;
using OrderService.Messaging;
using OrderService.Models;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;
using Testcontainers.RabbitMq;

namespace Integration.Tests;

public class OrderCreatedFlowTests : IAsyncLifetime
{
    private readonly RabbitMqContainer _rabbitMqContainer;
    private const string QueueName = "orders";

    public OrderCreatedFlowTests()
    {
        _rabbitMqContainer = new RabbitMqBuilder("rabbitmq:3-management")
            .WithUsername("guest")
            .WithPassword("guest")
            .Build();
    }

    public async Task InitializeAsync() =>
        await _rabbitMqContainer.StartAsync();

    public async Task DisposeAsync() =>
        await _rabbitMqContainer.DisposeAsync();

    [Fact]
    public async Task Publicar_UmaMensagem_DeveChegar_NaFila()
    {
        var host = _rabbitMqContainer.Hostname;
        var port = _rabbitMqContainer.GetMappedPublicPort(5672);

        var publisher = await RabbitMqPublisher.CreateAsync(host, port);

        var mensagem = new OrderCreatedMessage
        {
            OrderId = Guid.NewGuid(),
            CustomerName = "Teste Integração",
            TotalAmount = 100m,
            CreatedAt = DateTime.UtcNow
```

```
    };

    await publisher.PublishAsync(mensagem);

    var factory = new ConnectionFactory { HostName = host, Port = port };
    await using var connection = await factory.CreateConnectionAsync();
    await using var channel = await connection.CreateChannelAsync();

    await channel.QueueDeclareAsync(
        QueueName, durable: true, exclusive: false, autoDelete: false
    );

    var tcs = new TaskCompletionSource<OrderCreatedMessage>();
    var consumer = new AsyncEventingBasicConsumer(channel);

    consumer.ReceivedAsync += (_, args) =>
    {
        var json = Encoding.UTF8.GetString(args.Body.ToArray());
        var received = JsonSerializer.Deserialize<OrderCreatedMessage>(json)!;
        tcs.SetResult(received);
        return Task.CompletedTask;
    };

    await channel.BasicConsumeAsync(QueueName, autoAck: true, consumer: consumer);

    var mensagemRecebida = await tcs.Task.WaitAsync(TimeSpan.FromSeconds(5));

    Assert.Equal(mensagem.OrderId, mensagemRecebida.OrderId);
    Assert.Equal(mensagem.CustomerName, mensagemRecebida.CustomerName);
    Assert.Equal(mensagem.TotalAmount, mensagemRecebida.TotalAmount);

    await publisher.DisposeAsync();
}

[Fact]
public async Task Publicar_MultiplasMensagens_TodasDevemChegar_NaFila()
{
    var host = _rabbitMqContainer.Hostname;
    var port = _rabbitMqContainer.GetMappedPublicPort(5672);

    var publisher = await RabbitMqPublisher.CreateAsync(host, port);
    var mensagens = Enumerable.Range(1, 3).Select(i => new OrderCreatedMessage
    {
```

```
            OrderId = Guid.NewGuid(),
            CustomerName = $"Cliente {i}",
            TotalAmount = i * 10m,
            CreatedAt = DateTime.UtcNow
        }).ToList();

        foreach (var msg in mensagens)
            await publisher.PublishAsync(msg);

        var factory = new ConnectionFactory { HostName = host, Port = port };
        await using var connection = await factory.CreateConnectionAsync();
        await using var channel = await connection.CreateChannelAsync();

        await channel.QueueDeclareAsync(
            QueueName, durable: true, exclusive: false, autoDelete: false
        );

        var recebidas = new List<OrderCreatedMessage>();
        var tcs = new TaskCompletionSource();
        var consumer = new AsyncEventingBasicConsumer(channel);

        consumer.ReceivedAsync += (_, args) =>
        {
            var json = Encoding.UTF8.GetString(args.Body.ToArray());
            recebidas.Add(JsonSerializer.Deserialize<OrderCreatedMessage>(json)!);
            if (recebidas.Count == mensagens.Count) tcs.SetResult();
            return Task.CompletedTask;
        };

        await channel.BasicConsumeAsync(QueueName, autoAck: true, consumer: consumer);
        await tcs.Task.WaitAsync(TimeSpan.FromSeconds(5));

        Assert.Equal(3, recebidas.Count);
        Assert.All(mensagens, m =>
            Assert.Contains(recebidas, r => r.OrderId == m.OrderId)
        );

        await publisher.DisposeAsync();
    }
}
```

Explicando os conceitos novos:

IAsyncLifetime

```
public class OrderCreatedFlowTests : IAsyncLifetime
```

`IAsyncLifetime` é uma interface do xUnit que fornece dois métodos:

- `InitializeAsync` : roda **antes** de cada teste — usamos para subir o container.
- `DisposeAsync` : roda **depois** de cada teste — usamos para derrubar o container.

TaskCompletionSource

```
var tcs = new TaskCompletionSource<OrderCreatedMessage>();
```

Isso cria um `Task` que podemos **completar manualmente**. Quando a mensagem chega no consumer, chamamos `tcs.SetResult(received)` e o `await tcs.Task` no teste é liberado. É como criar uma promessa: "vou te entregar um resultado quando ele chegar".

GetMappedPublicPort

```
var port = _rabbitMqContainer.GetMappedPublicPort(5672);
```

O Testcontainers mapeia a porta 5672 do container para uma **porta aleatória** no host. Isso evita conflitos se você já tiver outro RabbitMQ rodando na porta 5672.

10. Etapa 7 — Rodando o Projeto Completo

10.1 Subir o ambiente com Docker Compose

Na raiz do projeto:

```
docker compose up --build
```

O que esse comando faz:

1. Constrói as imagens Docker dos dois serviços (OrderService e NotificationService).
2. Sobe o container do RabbitMQ.
3. Sobe os containers dos serviços.

4. Mostra os logs de todos os containers no terminal.

Espere até ver no terminal:

```
notification-service  | NotificationService pronto. Aguardando mensagens...
```

Isso significa que tudo está funcionando.

10.2 Criar um pedido

Abra **outro terminal** (não feche o Docker Compose) e envie um pedido:

PowerShell:

```
$body = @{
    customerName = "Joao Silva"
    totalAmount  = 150.90
} | ConvertTo-Json

Invoke-RestMethod -Method Post -Uri "http://localhost:5000/orders" `
  -ContentType "application/json" `
  -Body $body
```

> **Atenção:** não use aspas simples ao redor do JSON no PowerShell — pode causar erro 400. Use o `ConvertTo-Json` como mostrado acima.

curl (Git Bash ou Linux):

```
curl -X POST http://localhost:5000/orders \
  -H "Content-Type: application/json" \
  -d '{"customerName": "Joao Silva", "totalAmount": 150.90}'
```

Resposta esperada:

```
{
  "orderId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "status": "Pedido recebido e sendo processado"
}
```

10.3 O que observar nos logs

Volte ao terminal do Docker Compose. Você verá algo como:

```
order-service         | Pedido a1b2c3d4-... publicado no RabbitMQ
notification-service  | [OrderConsumer] Pedido a1b2c3d4-... → 3 notificações despachadas
notification-service  | [EMAIL] Enviado para joao.silva@example.com | Pedido a1b2c3d4-...
notification-service  | [PUSH] Enviado para device-a1b2c3d4 | Pedido a1b2c3d4-...
notification-service  | [RETRY 1/3] sms falhou: Operadora indisponível para '+55 11 9 0000-0000
notification-service  | [SMS] Enviado para +55 11 9 0000-0000 | Pedido a1b2c3d4-...
```

Como os handlers têm taxas de falha simuladas, você verá retries acontecendo naturalmente. Envie vários pedidos para ver o mecanismo de DLQ em ação.

10.4 Acessar o painel do RabbitMQ

Abra no navegador: http://localhost:15672

- **Usuário:** guest
- **Senha:** guest

No painel, vá em **Queues** para ver:

- `orders` : fila de pedidos (deve estar vazia se tudo foi consumido).
- `notifications` : fila principal de notificações.
- `notifications.retry.1` , `.2` , `.3` : filas de retry.
- `notifications.dlq` : mensagens que falharam após todas as tentativas.

10.5 Rodar os testes

```
dotnet test
```

Isso executa tanto os testes unitários quanto os de integração. Os testes de integração vão subir um container RabbitMQ temporário automaticamente (precisa do Docker rodando).

10.6 Parar o ambiente

No terminal do Docker Compose, pressione `Ctrl+C` e depois:

```
docker compose down
```

Isso derruba todos os containers e limpa os recursos.

11. Glossário

Termo	Definição
AMQP	Advanced Message Queuing Protocol — protocolo de mensageria usado pelo RabbitMQ
API	Application Programming Interface — interface para comunicação entre programas
Async/Await	Padrão de programação assíncrona em C# — permite esperar por operações longas sem travar
Backoff Exponencial	Estratégia de retry onde o tempo de espera cresce exponencialmente entre tentativas
Container	Ambiente isolado e portátil para rodar aplicações (gerenciado pelo Docker)
Consumer	Programa que lê e processa mensagens de uma fila
DI (Dependency Injection)	Padrão onde objetos recebem suas dependências externamente
DLQ (Dead Letter Queue)	Fila para mensagens que falharam e não serão mais retentadas
DTO (Data Transfer Object)	Objeto usado apenas para transportar dados entre camadas
Exchange	Componente do RabbitMQ que roteia mensagens para filas
GUID	Globally Unique Identifier — identificador único universal (ex: `a1b2c3d4-e5f6-...`)
HTTP 202 Accepted	"Recebi seu pedido, mas ainda estou processando"
Idempotência	Propriedade de uma operação que, executada múltiplas vezes, produz o mesmo resultado

Termo	Definição
JSON	JavaScript Object Notation — formato leve de troca de dados
Microserviço	Serviço pequeno e independente que faz uma coisa bem feita
Mock	Objeto falso usado em testes para simular dependências
Multi-stage build	Técnica de Dockerfile com múltiplos estágios para imagens menores
Producer	Programa que envia mensagens para uma fila
Queue (Fila)	Estrutura onde mensagens ficam armazenadas até serem consumidas
Record	Tipo em C# otimizado para dados imutáveis
REST	Representational State Transfer — estilo arquitetural para APIs HTTP
Routing Key	Chave usada pela exchange para decidir para qual fila enviar a mensagem
SDK	Software Development Kit — kit de ferramentas para desenvolver aplicações
Serialização	Converter um objeto em texto (JSON) ou bytes para transmissão
Singleton	Padrão onde apenas uma instância de uma classe existe durante toda a vida da aplicação
Strategy	Padrão de design onde algoritmos são encapsulados em classes intercambiáveis
TTL	Time To Live — tempo máximo que uma mensagem pode ficar em uma fila
Worker	Serviço em segundo plano que roda continuamente executando tarefas
xUnit	Framework de testes para .NET
YAML	Yet Another Markup Language — formato de configuração usado pelo Docker Compose

12. Diagrama Completo da Arquitetura

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                          MICROSERVICES-DEMO — VISÃO GERAL                 ║
╚═══════════════════════════════════════════════════════════════════════════╝


  ┌──────────────────┐
  │   Cliente HTTP   │
  │  (Postman/curl/  │
  │   navegador)     │
  └──────────────────┘
          │
          │ POST /orders
          │ { customerName, totalAmount }
          ▼
  ┌──────────────────┐
  │   OrderService   │   ◄── ASP.NET Web API (.NET 8)
  │   (porta 5000)   │       Container Docker
  │                  │
  │  OrdersController│
  │       │          │
  │       ▼          │
  │  RabbitMqPublisher│─── Serializa para JSON
  └──────────────────┘     e publica
          │
          │
          │ exchange: "" (default)
          │ routingKey: "orders"
          ▼
╔═══════════════════════════════════════════════════════════════════════════╗
║                              RABBITMQ                                     ║
║                        (Container Docker)                                 ║
║                                                                           ║
║  ┌──────────┐                                                             ║
║  │  orders  │ ◄── fila de pedidos (durable)                               ║
║  └──────────┘                                                             ║
║       │                                                                   ║
║       │                                                                   ║
║       │ (consumido pelo OrderConsumer)                                    ║
║       │                                                                   ║
║  ┌────────────────────────┐     ┌────────────────────────┐               ║
║  │  notifications.exchange │     │ notifications.retry.   ║               ║
║  │  (DIRECT)              │     │ exchange (DIRECT)      │               ║
║  └────────────────────────┘     └────────────────────────┘               ║
║             │                              │                              ║
║             │ "notification"               │ "retry.1/2/3"                ║
```

```
│              ▼                          ▼                                │
│  ┌──────────────────┐     ┌──────────────────────────┐                 │
│  │  notifications   │◄────│  notifications.retry.1 (5s)  │              │
│  │  (fila principal)│◄────│  notifications.retry.2 (15s) │              │
│  │                  │◄────│  notifications.retry.3 (45s) │              │
│  └──────────────────┘     └──────────────────────────┘                 │
│                                                                         │
│  ┌──────────────────┐                                                   │
│  │  notifications.dlq │ ◄── mensagens que esgotaram retries             │
│  └──────────────────┘                                                   │
└─────────────────────────────────────────────────────────────────────────┘
          │
          │ (consumido pelo NotificationConsumer)
          ▼
┌──────────────────────────────────────────────┐
│           NotificationService                 │ ◄── Worker (.NET 8)
│           (Container Docker)                  │     Container Docker
│                                               │
│  ┌──────────────┐  ┌──────────────────────┐  │
│  │OrderConsumer │  │  NotificationConsumer  │  │
│  │  (1 pedido   │  │  (5 notificações em    │  │
│  │   = 3 notif) │  │   paralelo, retry,     │  │
│  └──────────────┘  │   DLQ, idempotência)   │  │
│                    └──────────────────────┘    │
│                                               │
│  Handlers (Strategy Pattern):                 │
│  ┌──────────────┬──────────────┬──────────────┐ │
│  │ Email (30%   │ Push (20%    │ SMS (40%     │ │
│  │  falha sim.) │  falha sim.) │  falha sim.) │ │
│  └──────────────┴──────────────┴──────────────┘ │
│                                               │
│  Serviços de suporte:                         │
│  ┌──────────────────┬──────────────────────┐  │
│  │ IdempotencyService│ MetricsService       │  │
│  │ (evita duplicatas)│ (contadores + log 30s)│ │
│  └──────────────────┴──────────────────────┘  │
└──────────────────────────────────────────────┘
```

13. Dicas e Próximos Passos

O que você aprendeu

- Como criar uma **API Web** com ASP.NET Core.
- Como criar um **Worker Service** para processamento em segundo plano.
- Como usar **RabbitMQ** para comunicação assíncrona entre serviços.
- Como implementar **retry com backoff exponencial** e **Dead Letter Queue**.
- Como garantir **idempotência** no processamento de mensagens.
- Como **containerizar** aplicações .NET com Docker.
- Como **orquestrar** múltiplos containers com Docker Compose.
- Como escrever **testes unitários** com xUnit e Moq.
- Como escrever **testes de integração** com Testcontainers.
- Como usar **Injeção de Dependência** e o padrão **Strategy**.

Ideias para evoluir o projeto

1. **Adicionar um banco de dados**: use Entity Framework Core com PostgreSQL para persistir os pedidos.
2. **Implementar idempotência com Redis**: substitua o `ConcurrentDictionary` por Redis para funcionar com múltiplas instâncias.
3. **Adicionar health checks**: implemente `/health` endpoints para monitorar a saúde dos serviços.
4. **Usar OpenTelemetry**: adicione tracing distribuído para rastrear uma requisição por todos os serviços.
5. **Adicionar API Gateway**: coloque um gateway (como YARP ou Ocelot) na frente dos serviços.
6. **Implementar autenticação**: adicione JWT para proteger o endpoint de pedidos.
7. **Adicionar Swagger/OpenAPI**: documente a API com Swagger UI.
8. **Escalar horizontalmente**: suba múltiplas instâncias do NotificationService e veja o RabbitMQ distribuindo as mensagens.
9. **Adicionar CI/CD**: configure GitHub Actions para rodar os testes automaticamente a cada push.
10. **Separar a topologia**: crie um serviço dedicado para gerenciar a topologia do RabbitMQ.

Comandos úteis de referência rápida

Comando	O que faz
`dotnet new sln -n NomeDaSolucao`	Cria uma solution
`dotnet new webapi -n NomeDoProjeto`	Cria um projeto de API Web

Comando	O que faz
`dotnet new worker -n NomeDoProjeto`	Cria um projeto Worker
`dotnet new xunit -n NomeDoProjeto`	Cria um projeto de testes xUnit
`dotnet sln add Projeto/Projeto.csproj`	Adiciona um projeto à solution
`dotnet add package NomeDoPacote`	Instala um pacote NuGet
`dotnet add reference ../Outro/Outro.csproj`	Adiciona referência entre projetos
`dotnet build`	Compila a solution
`dotnet test`	Roda todos os testes
`dotnet run --project NomeDoProjeto`	Roda um projeto específico
`docker compose up --build`	Sobe o ambiente com Docker
`docker compose down`	Derruba o ambiente
`docker compose logs -f nome-servico`	Acompanha logs de um serviço

Parabéns! Se você chegou até aqui, você construiu um sistema de microserviços completo do zero. Isso não é pouca coisa. Agora use esse conhecimento como base para construir seus próprios projetos.

Lembre-se: a melhor forma de aprender é fazendo. Não tenha medo de errar — cada erro é uma aula.